

```
In [1]: import pandas as pd
import numpy as np

import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image

for i in [pd, np, mpl, sns]:
    print(i.__name__, i.__version__)
```

```
pandas 2.1.4
numpy 1.26.4
matplotlib 3.8.0
seaborn 0.12.2
```

1-4 데이터 시각화

- 패턴 및 관계 파악, 효과적인 의사 소통
1. 추세 및 패턴 시각화
 2. 이상치 탐지
 3. 시각화를 통한 변수간 관계 파악

```
In [2]: Image('22.png')
```

Out[2]: **0. 데이터셋 소개**

Space Titanic

variable name	description
PassengerId	A unique Id for each passenger. Each Id takes the form gggg_pp where gggg indicates a group the passenger is travelling with and pp is their number within the group. People in a group are often family members, but not always.
HomePlanet	The planet the passenger departed from, typically their planet of permanent residence.
CryoSleep	Indicates whether the passenger elected to be put into suspended animation for the duration of the voyage. Passengers in cryosleep are confined to their cabins.
Cabin	The cabin number where the passenger is staying. Takes the form deck/num/side, where side can be either P for Port or S for Starboard.
Destination	The planet the passenger will be debarking to.
Age	The age of the passenger.
VIP	Whether the passenger has paid for special VIP service during the voyage.
RoomService, FoodCourt, ShoppingMall, Spa, VRDeck	Amount the passenger has billed at each of the Spaceship Titanic's many luxury amenities.
Name	The first and last names of the passenger.
Transported	Whether the passenger was transported to another dimension. This is the target, the column you are trying to predict.

```
In [2]: df_space = pd.read_csv('data/space_titanic.csv', index_col='PassengerId')
df_space.head()
```

Out[2]:

	HomePlanet	CryoSleep	Cabin	Destination	Age	VIP	RoomService	FoodCourt	ShoppingMall	Spa	VRDeck	Nar
PassengerId												
0001_01	Europa	False	B/0/P	TRAPPIST-1e	39.0	False	0.0	0.0	0.0	0.0	0.0	Maha Ofracci
0002_01	Earth	False	F/0/S	TRAPPIST-1e	24.0	False	109.0	9.0	25.0	549.0	44.0	Juan Vir
0003_01	Europa	False	A/0/S	TRAPPIST-1e	58.0	True	43.0	3576.0	0.0	6715.0	49.0	Alta Suse
0003_02	Europa	False	A/0/S	TRAPPIST-1e	33.0	False	0.0	1283.0	371.0	3329.0	193.0	Sola Suse
0004_01	Earth	False	F/1/S	TRAPPIST-1e	16.0	False	303.0	70.0	151.0	565.0	2.0	W Santantir

데이터 전처리

1. PassengerID를 '_'로 나누어 첫번째 변수는 PassengerGrp,

두 번째는 'GrpNo'로 추가합니다.

2. GrpNo는 정수형으로 형변환 합니다.

In [3]:

```
'''
1. `df_space` 데이터프레임을 기존 데이터프레임에 추가하는데,
   이 때 `PassengerId`를 기준으로 인덱스를 다시 만들어서 활용합니다.
2. `df_space.index.to_frame()`은 데이터프레임의 인덱스를
   열로 바꾼 새로운 데이터프레임을 생성합니다.
3. `.str.split('_', expand=True)`은 `PassengerId`를
   언더스코어(`_`)를 기준으로 분할합니다.
   그리고 이를 여러 열로 나누어 새로운 데이터프레임으로 만듭니다.
4. `.rename(columns={0: 'PassengerGrp', 1: 'GrpNo'})`는 새로 생성된
   열의 이름을 변경합니다.
'''
```

분할된 첫 번째 열은 'PassengerGrp'로, 두 번째 열은 'GrpNo'로 지정합니다.

5. ``.assign(GrpNo=lambda x: x['GrpNo'].astype(int))``는 'GrpNo' 열을 정수형으로 변환하여 할당합니다.

6. 마지막으로, ``pd.concat()`` 함수를 사용하여 원래의 ``df_space`` 데이터프레임과 새로 생성된 열을 합칩니다.

이 코드는 'PassengerId' 열의 값을 기준으로 그룹을 나누고, 그룹 번호를 추출하여 'PassengerGrp'와 'GrpNo' 열을 추가하는 것입니다.

'''

```
# pd.Series.str.split에서 expand=True로 설정하면
# 분리된 문자열들을 리스트에 모아 저장하지 않고,
# 순서에 따라 컬럼으로 분리해서 뽑아냅니다.
df_space = pd.concat([
    df_space,
    df_space.index.to_frame()['PassengerId'].str.split('_', expand=True).rename(
        columns={0: 'PassengerGrp', 1: 'GrpNo'}
    ).assign(
        GrpNo = lambda x: x['GrpNo'].astype(int)
    ),
], axis=1)
```

In [4]: `df_space.head()`

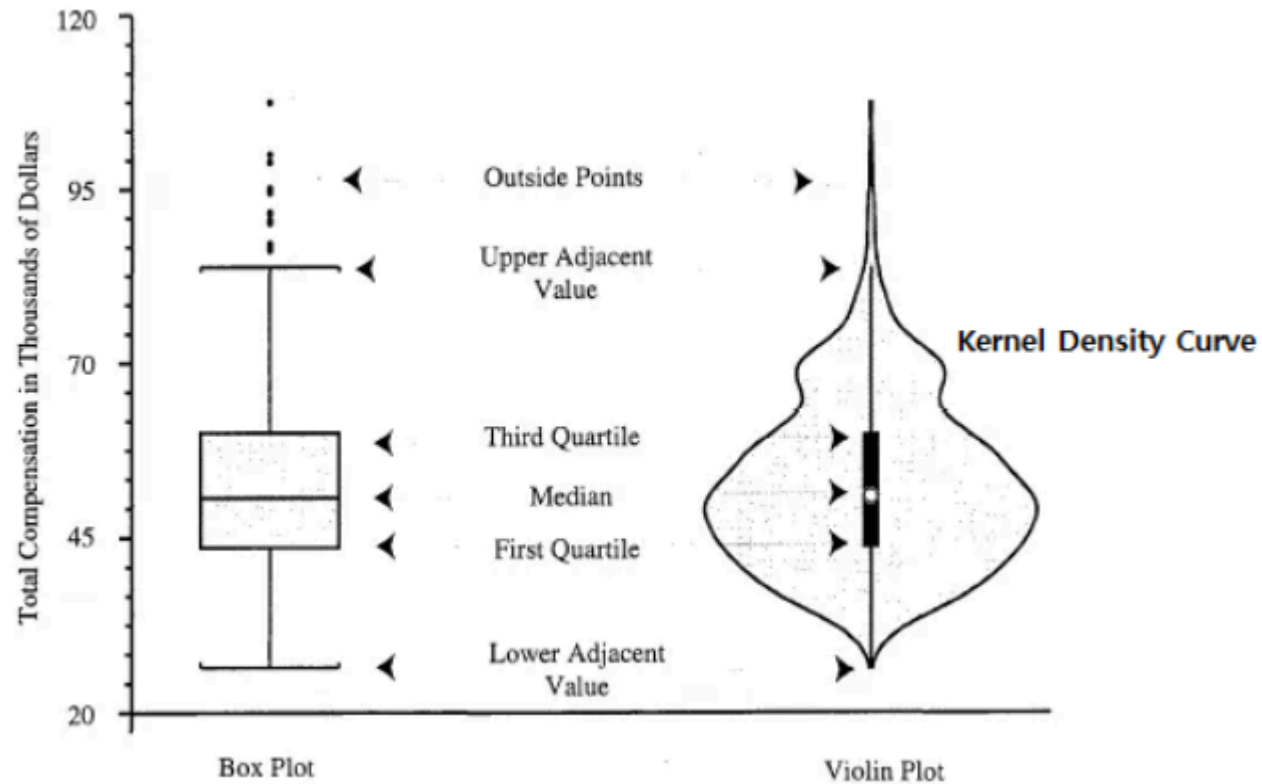
Out[4]:

	HomePlanet	CryoSleep	Cabin	Destination	Age	VIP	RoomService	FoodCourt	ShoppingMall	Spa	VRDeck	Nar
PassengerId												
0001_01	Europa	False	B/0/P	TRAPPIST-1e	39.0	False	0.0	0.0	0.0	0.0	0.0	Maha Ofracci
0002_01	Earth	False	F/0/S	TRAPPIST-1e	24.0	False	109.0	9.0	25.0	549.0	44.0	Juan Vir
0003_01	Europa	False	A/0/S	TRAPPIST-1e	58.0	True	43.0	3576.0	0.0	6715.0	49.0	Alta Suse
0003_02	Europa	False	A/0/S	TRAPPIST-1e	33.0	False	0.0	1283.0	371.0	3329.0	193.0	Sola Suse
0004_01	Earth	False	F/1/S	TRAPPIST-1e	16.0	False	303.0	70.0	151.0	565.0	2.0	W Santantir

1. 바이올린 플롯(Viloin Plot)

In [2]: Image('23.png')

Out[2]:



특징

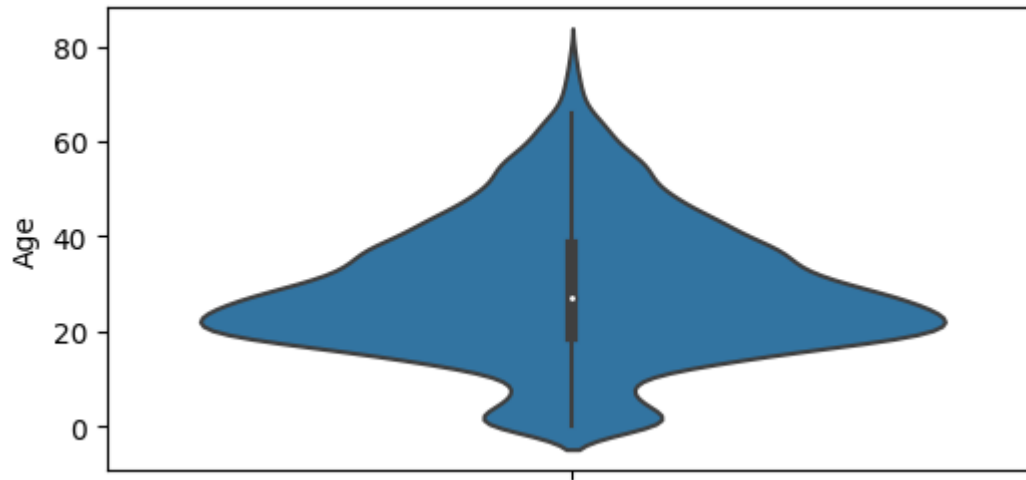
1. 연속형 데이터의 분포와 사분위 통계 정보를 출력합니다.
 - 박스 플롯과 커널 밀도 플롯이 결합한 형태의 출력물을 제공합니다.
2. 범주형 데이터와 결합하여 차트를 구성할 수 있습니다.
 - 이진형 변수와 결합하여 중심축을 기준으로 좌우를 구분하여 나타낼 수 있습니다.

- 범주형 변수를 축에 포함시켜, 수준별 바이올린 차트를 출력하도록 구성할 수 있습니다.

[Ex.1]

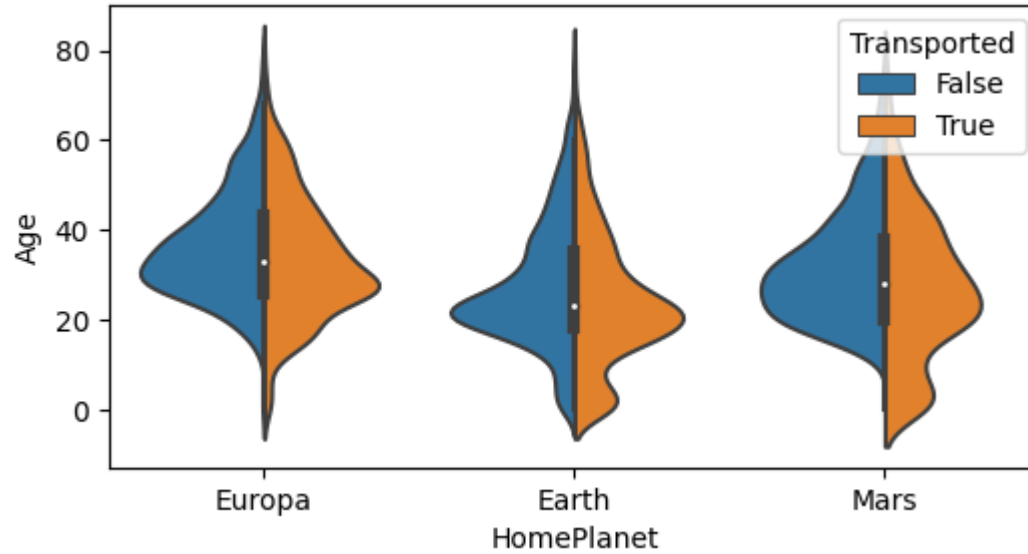
df_space의 Age를 바이올린 플롯으로 출력합니다.

```
In [5]: plt.figure(figsize=(6, 3))  
sns.violinplot(data=df_space, y='Age')  
plt.show()
```

**[Ex.2]**

df_space에서 중심축 좌우를 Transported로 구분하고
x축은 HomePlanet으로, y축은 Age로 하여 바이올린 플롯으로 출력합니다.

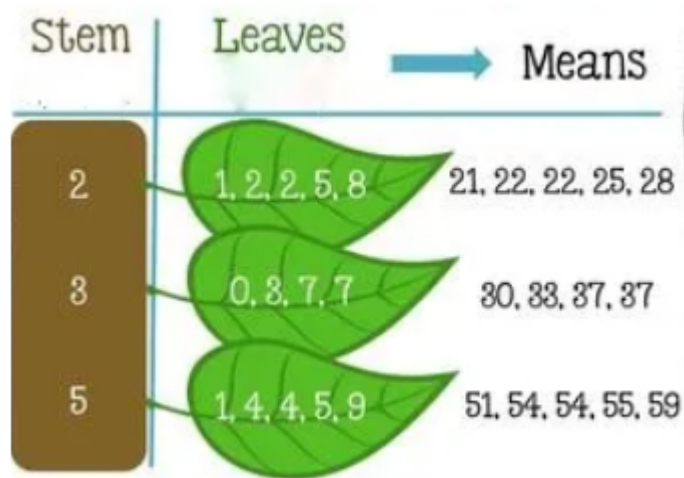
```
In [6]: plt.figure(figsize=(6, 3))  
sns.violinplot(data=df_space, x='HomePlanet', y='Age', split=True,  
              hue='Transported')  
plt.show()
```



2. 줄기 잎 플롯 (Stem and Leaf Plot)

```
In [3]: Image('24.png')
```


Out[3]:



특징

- 데이터의 분포와 동시에 상세 값을 표시할 수 있습니다.

[Ex.3]

Stem and Leaf Plot을 이용하여 다음 데이터를 출력해봅니다.

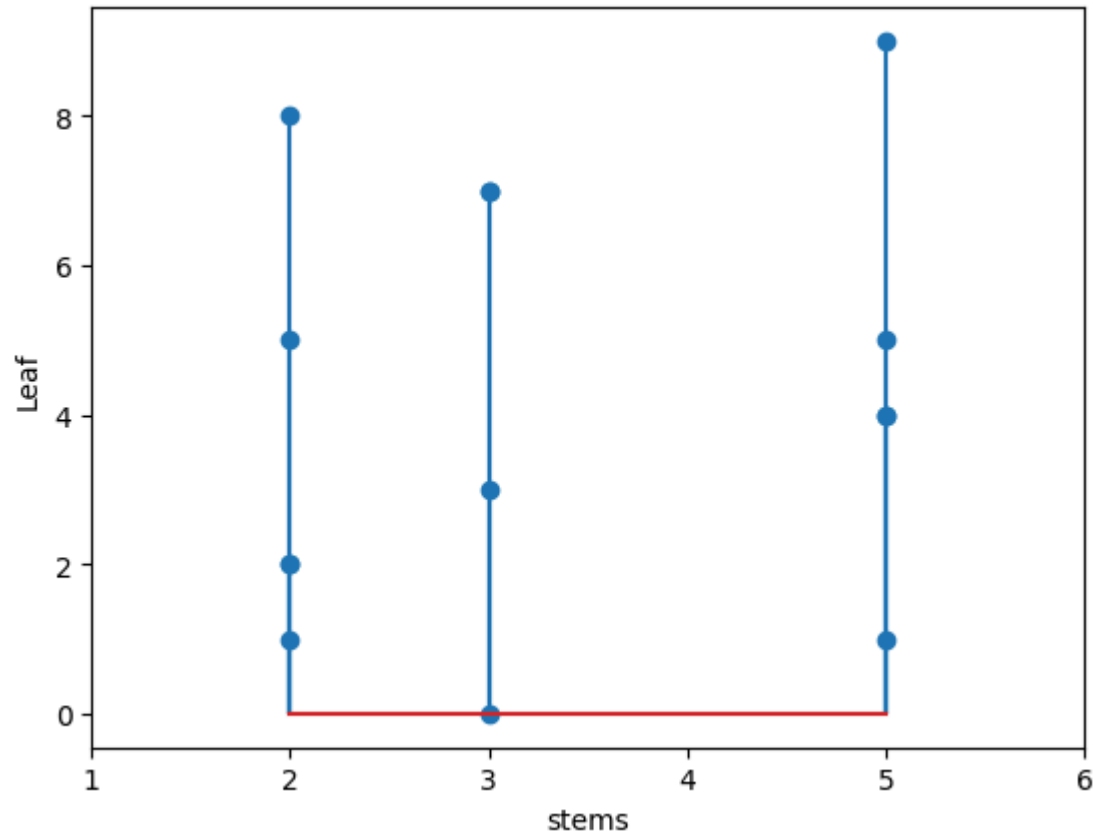
```
data = np.array([21, 22, 22, 25, 28, 30, 33, 37, 37, 51, 54, 54, 55, 59])
```

```
In [7]: data = np.array([21, 22, 22, 25, 28, 30, 33, 37, 37, 51, 54, 54, 55, 59])
        stems = data // 10

        plt.ylabel('Leaf')
        plt.xlabel('stems')
        plt.xlim(1, 6)

        plt.stem(
            data // 10, # Stem
            data % 10, # Leaf
```

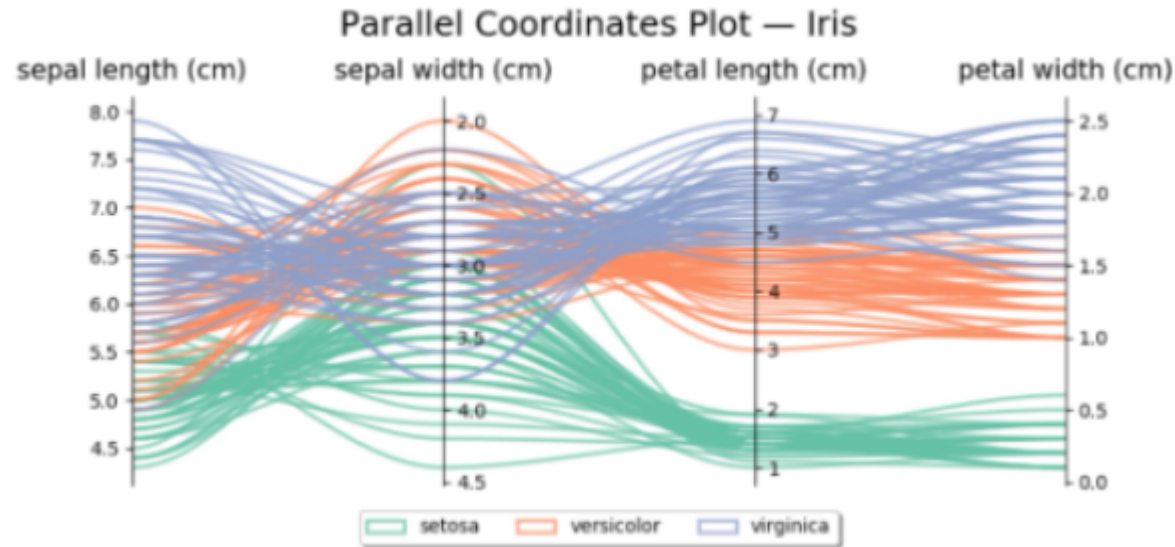
```
)  
plt.show()
```



평행 좌표 그림 (Parallel Coordinates)

```
In [4]: Image('25.png')
```

Out[4]:



특징

1. 다차원 데이터의 패턴이나 군집을 직관적으로 보여줍니다.
 - 변수는 수직선으로 표현이 되고, 값은 수직선 위에 점으로 표시합니다.
 - 동일 레코드에 수직선에 표시된 이웃점들을 선으로 연결하여 표시합니다.
 - 색상이나 점의 모양으로 표본의 소속 군집을 표현합니다.
2. 데이터가 너무 많으면 지나치게 많은 선들이 겹쳐지게 되어 파악이 어려워집니다.
3. 효과적인 출력을 위한 변수의 출력 순서를 정하기 어렵습니다.

[Ex.4]

CryoSleep이 False인 행들에서 PassengerGrp과 HomePlanet별로

RoomService, FoodCourt, ShoppingMall, Spa, VRDeck의 합계를 내어

df_spend 데이터프레임을 만듭니다.

RoomService와 FoodCourt, ShoppingMall, Spa, VRDeck을 $\log(X + 1)$ 변환하여,

변수의 폭을 줄여서 그래프로 출력이 용이하도록 합니다.

HomePlanet 별로 색상을 구분한 Parellel Coordindates를 출력합니다.

In [8]:

```
'''
코드는 주어진 데이터프레임인 `df_space`에서 'CryoSleep' 열이 False인
행들을 선택하고, 이를 그룹화하여 각 그룹별로
'PassengerGrp'와 'HomePlanet'에 따른 지출 항목들의 합을 구하는 과정입니다.

1. `df_space.loc[df_space['CryoSleep'] == False]`은 'CryoSleep' 열이
False인 행들을 선택합니다. 이는 CryoSleep(얼음 침대에 눕는 상태)가 아닌
스페이스 여행객들을 선택하는 것입니다.
2. `.groupby(['PassengerGrp', 'HomePlanet'], as_index=False)`은
'PassengerGrp'와 'HomePlanet' 열을 기준으로 데이터를 그룹화합니다.
이때 `as_index=False`는 그룹화된 열을 인덱스로 사용하지 않고
새로운 인덱스를 생성하는 것을 의미합니다.
3. `[spend_cols].sum()`은 선택된 그룹에 대해
'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck' 열의
값을 합산합니다. 이렇게 하면 각 그룹의 'PassengerGrp'와 'HomePlanet'에
따른 총 지출이 계산됩니다.
4. `np.log(df_spend[spend_cols] + 1)`은 각 지출 항목에 대해 자연로그를 취합니다.
`np.log()` 함수는 입력 값의 자연로그를 계산하며,
`+ 1`을 추가하여 로그 값이 음수가 되지 않도록 보장합니다.

결과적으로, 코드는 스페이스 여행객의 그룹 및 출신 행성별로
각 지출 항목의 로그 변환된 총 합을 계산합니다.
'''

spend_cols = ['RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck']
```

```
df_spend = df_space.loc[df_space['CryoSleep'] == False]\
    .groupby(['PassengerGrp', 'HomePlanet'],
             as_index=False)[spend_cols].sum()

df_spend[spend_cols] = np.log(df_spend[spend_cols] + 1)

df_spend
```

Out[8]:

	PassengerGrp	HomePlanet	RoomService	FoodCourt	ShoppingMall	Spa	VRDeck
0	0001	Europa	0.000000	0.000000	0.000000	0.000000	0.000000
1	0002	Earth	4.700480	2.302585	3.258097	6.309918	3.806662
2	0003	Europa	3.784190	8.488794	5.918894	9.214830	5.493061
3	0004	Earth	5.717028	4.262680	5.023881	6.338594	1.098612
4	0005	Earth	0.000000	6.182085	0.000000	5.676754	0.000000
...	...	...	...	...	...	...	...
4349	9272	Earth	5.789960	5.505332	6.492240	5.342334	5.799093
4350	9275	Europa	0.693147	7.044905	0.000000	3.931826	3.555348
4351	9276	Europa	0.000000	8.827615	0.000000	7.404888	4.317488
4352	9279	Earth	0.000000	0.000000	7.535297	0.693147	0.000000
4353	9280	Europa	4.844187	8.654866	0.000000	5.869297	8.085795

4354 rows × 7 columns

In [9]:

```
'''
코드는 주어진 데이터프레임 `df_spend`를 사용하여 병렬 좌표 그래프를 생성합니다.
병렬 좌표 그래프는 다차원 데이터를 시각화하는 데 사용되며,
각 변수가 수직선으로 나타나고 이러한 수직선들이 서로 연결되어 병렬로 배열됩니다.

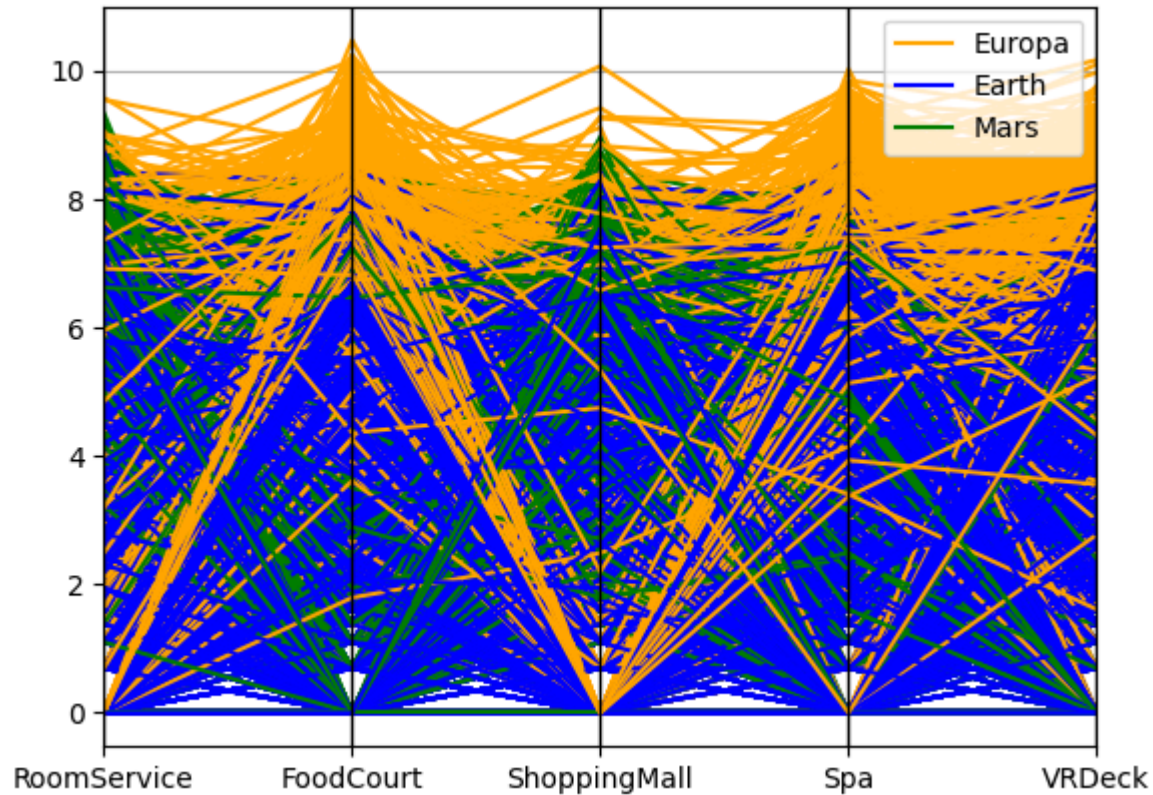
1. `pd.plotting.parallel_coordinates()` 함수는 Pandas에서 제공하는
   병렬 좌표 그래프를 생성하는 함수입니다.
```

2. 첫 번째 인자인 `df_spend`는 시각화할 데이터프레임을 나타냅니다.
3. 두 번째 인자인 `'HomePlanet'`는 그래프에서 각 그룹을 구분할 기준 변수를 나타냅니다. 여기서는 'HomePlanet' 열을 기준으로 병렬 좌표 그래프를 생성합니다.
4. 세 번째 인자인 `spend_cols`는 시각화할 변수들의 이름을 포함하는 리스트입니다. 여기서는 'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck' 열이 선택됩니다.
5. 네 번째 인자인 `color=['orange', 'blue', 'green']`는 그룹을 나타내는 색상을 지정합니다. 여기서는 'HomePlanet'의 값이 서로 다른 그룹을 각각 주황색, 파란색, 녹색으로 표시합니다.

이렇게 하면 주어진 데이터프레임에 대해 'HomePlanet' 열의 값에 따라 다른 색으로 구분된 병렬 좌표 그래프가 생성됩니다.
이 그래프를 통해 각 그룹의 지출 항목 간의 패턴이 시각적으로 비교 및 분석할 수 있습니다.

...

```
pd.plotting.parallel_coordinates(df_spend, 'HomePlanet', spend_cols,  
                                color=['orange', 'blue', 'green'])  
plt.show()
```



[Ex.5]

HomePlanet 별로 20개의 표본을 뽑아 출력해 봅니다.

In [10]:

```
'''
코드는 주어진 데이터프레임 `df_spend`를 사용하여 병렬 좌표 그래프를 생성합니다.
그러나 이전 코드와는 다르게 각 행성별로 모든 데이터를 사용하는 대신,
각 행성별로 무작위로 선택된 샘플을 사용하여 시각화를 수행합니다.

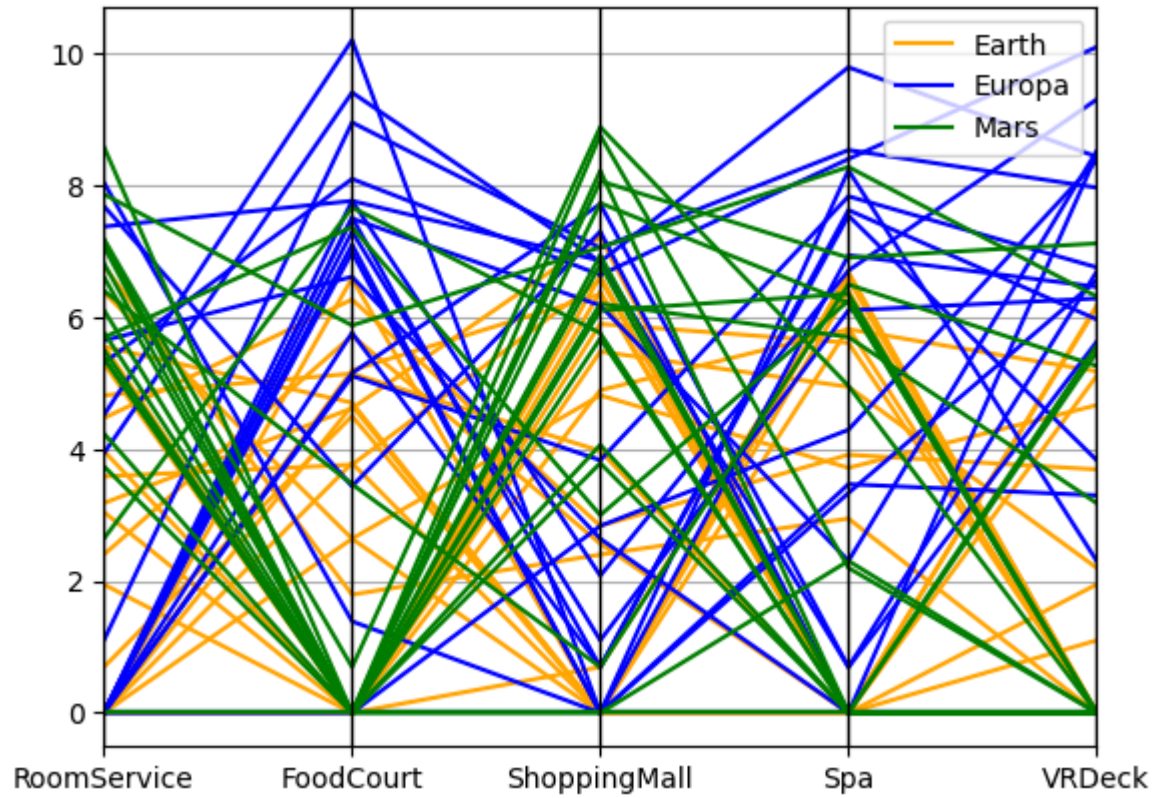
1. `df_spend.groupby('HomePlanet', as_index=False)`은 'HomePlanet' 열을 기준으로
   데이터프레임을 그룹화합니다. 이때 `as_index=False`는 그룹화된 열을
   인덱스로 사용하지 않고 새로운 인덱스를 생성하는 것을 의미합니다.
'''
```

2. `.apply(lambda x: x.sample(n=20, random_state=123))`는 각 그룹에 대해 무작위로 20개의 샘플을 선택합니다. 이때 `random_state=123`은 재현 가능한 결과를 얻기 위해 난수 발생을 위한 시드(seed)를 설정하는 것입니다.
3. 이렇게 선택된 샘플을 포함하는 새로운 데이터프레임을 생성하고, 이를 `pd.plotting.parallel_coordinates()` 함수에 전달하여 병렬 좌표 그래프를 생성합니다.
4. `'HomePlanet'`은 각 그룹을 나타내는 변수로 사용됩니다.
5. `spend_cols`는 시각화할 변수들의 이름을 포함하는 리스트입니다.
6. `color=['orange', 'blue', 'green']`는 각 그룹을 나타내는 색상을 지정합니다.

결과적으로, 이 코드는 각 행성별로 무작위로 선택된 샘플을 사용하여 지출 항목 간의 패턴을 시각적으로 비교할 수 있는 병렬 좌표 그래프를 생성합니다.

```
pd.plotting.parallel_coordinates(
    df_spend.groupby('HomePlanet', as_index=False) \
        .apply(lambda x: x.sample(n=20, random_state=123)),
    'HomePlanet', spend_cols, color=['orange', 'blue', 'green']
)

plt.show()
```

In []: